

SIMATS ENGINEERING ASSESSMENT TOOL 3

COURSE NAME: MLA03

COURSE CODE: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO1: Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems.(BL2, BL3)

Assessment Tool Weightage: 30%

Dr Dumpala Prasanth

QUESTIONS: 10

Total Marks:50

Annexure A

Constraint - Based Problem-Solving

Duration: 2Hrs

1. Develop a Python-based Reinforcement Learning model using the OpenAI Gym environment to solve a Grid World navigation problem where the agent must reach the goal while avoiding obstacles. Explain how the state space, action space, reward function, and constraints are defined.

(10 Marks)

2. Implement an ϵ -greedy Multi-Armed Bandit algorithm in Python to maximize rewards under a limited budget of 500 trials. Compare the effect of different ϵ values (0.1, 0.3, and 0.5) on exploration, exploitation, and constraint satisfaction.

(10 Marks)

3. Design a Markov Decision Process (MDP) for an autonomous robot that must reach a destination while minimizing energy consumption. Define the states, actions, transition probabilities, rewards, and energy constraints.

(10 Marks)

4. Using Python and TensorFlow/Keras, implement a value iteration algorithm to determine the optimal policy for a constrained grid environment where certain

COI-AT3

① Reinforcement learning using openAI Gym for Grid World Navigation.

Objective:

The objective of this experiment is to develop a Reinforcement Learning model using the open AI Gym environment to solve a Grid World Navigation problem.

State space

The state space represents all possible positions of the agent within the grid. For a 4×4 grid, there are 16 possible states, and each state corresponds to a cell.

Action space

Move up, Move down, Move left, Move right.

Reward function:

Goal reached : +20, obstacle encountered : -15; Normal movement : -1; Invalid movement : -5

Advantages

- * Learns an optimal path without prior knowledge.
- * Suitable for autonomous navigation.
- * Can be extended to robotic path planning and game AI.

Result

The Reinforcement learning agent successfully learns the shortest and safest path to the goal while avoiding obstacles and maximizing cumulative rewards.

② ϵ -Greedy Multi-Armed Bandit Algorithm

Objective

The ϵ -greedy algorithm is a reinforcement learning technique used to balance exploration and exploitation.

The objective is to maximize the total reward obtained from multiple slot machines within a limited budget of 100 trials.

Working principle

- * Exploration: Randomly select an arm to gather new.
- * Exploitation: Select the arm with the highest estimated.
- * The value of ϵ determines the probability of exploration.

Comparison of ϵ values

ϵ value	<u>Exploration</u>	<u>Exploitation</u>	<u>Expected reward</u>
0.1	Low	High	Highest after learning.
0.3	Moderate	Moderate	Balanced performance.
0.5	High	Low	More exploration.

Advantages

- * Easy to implement.
- * Balances learning and decision-making.
- * Suitable for online recommendation system and advertisement.

Result

Among the tested values, $\epsilon = 0.1$ generally produces the highest cumulative reward because it exploits the performing arm more frequently after sufficient exploration.

Markov Decision Process (MDP) for an Autonomous Robot Objective

Design a Markov Decision Process (MDP) that enables an autonomous robot to reach a destination while minimizing energy consumption and avoiding unnecessary movements.

Components of MDP

States

Start, Corridor, charging station, destination, obstacle area.

Actions

- * Move Forward
- * Turn Left
- * Turn Right
- * Recharge
- * Stop

Reward Function

- * Destination reached : +100
- * Normal Movement : .2
- * Recharge : -5
- * Collision : -20
- * Battery exhausted : -100

Applications

- * Autonomous delivery robots
- * Warehouse automation.
- * Service robots.
- * Mobile robot navigation.

Result

The MDP identifies the optimal policy that minimizes energy consumption while maximizing the probability of successfully reaching the destination.

④ Value Iteration Algorithm

Objective

Value iteration is a dynamic programming technique used to compute the optimal policy for a Markov Decision process by repeatedly updating state values using the Bellman optimality equation.

Bellman Equation

$$V(s) = \max_a [R(s,a) + \gamma V(s')]]$$

$V(s)$ = value of state

γ = Discount factor

$R(s,a)$ = Immediate reward

s' = Next state.

Procedure

- Initialize all state values to zero.
- Update each state's value using the Bellman equation.
- Ignore restricted or blocked states.
- Repeat until the value converges.
- Extract the optimal policy from the final state values.

Advantage

- Finds an optimal policy.
- Guarantees convergence for finite MDPs.
- Widely used in robotics, AI, and navigation systems.

Result

The algorithm successfully computes the optimal value of each state and determines the best action while avoiding restricted states.

RL framework for a Warehouse Robot

Objective

Develop a Reinforcement Learning framework that enables a warehouse robot with a 30-minute battery capacity to complete the maximum number of deliveries while minimizing energy usage and avoiding collisions.

State Space

* Robot location, Remaining battery level, Delivery task status
distance to charging station, obstacle positions.

Action Space

* Pick package, Deliver package, Move forward, Left, Right
Recharge Battery, wait.

Constraints:

- * Battery life limited to 30-minutes.
- * Obstacles must be solved and avoided.
- * Deliveries should be completed before battery depletion.
- * The robot should return to the charging station whenever necessary.

Applications:

- * Smart warehouse.
- * E-commerce fulfillment centres.
- * Manufacturing industries.
- * Automated logistics system.

Result:

The reinforcement learning framework enables the warehouse robot to learn an efficient delivery policy that completes tasks, minimizes energy consumption, and ensures safe navigation within battery consumption.

states are restricted. Explain how Bellman equations are used to obtain the optimal policy.

(10 Marks)

5. A warehouse robot has a battery capacity of only 30 minutes. Design an RL framework that enables the robot to complete the maximum number of deliveries without exceeding the battery limit. Explain the constraints, policy, and reward design.

(10 Marks)

6. Implement a Reinforcement Learning agent that controls traffic signals at an intersection while ensuring that emergency vehicles receive immediate priority. Explain how exploration, exploitation, and policy learning are modified to satisfy the safety constraint.

(10 Marks)

7. Develop a Python program using Keras/TensorFlow to simulate a reinforcement learning agent for packet routing in a wireless network with limited bandwidth. Explain how bandwidth constraints influence the reward function and policy optimization.

(10 Marks)

8. Analyse an experimental comparison between random action selection and ϵ -greedy action selection in a Multi-Armed Bandit problem under a fixed time constraint. Record cumulative rewards and explain which strategy provides better performance.

(10 Marks)

9. Design a Reinforcement Learning framework for a delivery drone that must maximize successful deliveries while avoiding no-fly zones and maintaining battery limits. Explain the MDP formulation, Bellman equation, and policy learning process.

(10 Marks)

10. Implement a Reinforcement Learning solution using Python for a smart energy management system where electricity consumption must remain below a specified threshold. Explain how reward shaping, policy learning, and feasibility constraints improve decision-making.

(10 Marks)

Code of Conduct:

I G Saswanth (192425150) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided